ISO/IEC JTC1 SC22 WG21 P0490R0
Jens Maurer <Jens.Maurer@gmx.net>
Audience: Core Working Group
2016-11-11

P0490R0: Core language changes addressing National Body comments for CD C++17

This paper presents changes to the core language sections of the C++ Working Paper N4606 in response to National Body comments in response to the Committee Draft for C++17, N4604.

US 28: incorrect definition for principal constructor

This is a defect against C++14.

Change in 8.6 [dcl.init] paragraph 8:

> ~~An object that is value-initialized is deemed to be constructed and thus subject to provisions of this International Standard applying to "constructed" objects, objects "for which the constructor has completed," etc., even if no constructor is invoked for the object's initialization.~~

Add a new paragraph at the end of 8.6 [dcl.init]:

> **An object whose initialization has completed is deemed to be constructed, even if no constructor of the object's class is invoked for the initialization. [Note: Such an object might have been value-initialized or initialized by aggregate initialization (8.6.1 [dcl.init.aggr]) or by an inherited constructor (12.6.3 [class.inhctor.init]). -- end note]**

Change in 12.6.2 [class.base.init] paragraph 6:

> ... If a *mem-initializer-id* designates the constructor's class, it shall be the only *mem-initializer*; the constructor is a *delegating constructor*, and the constructor selected by the *mem-initializer* is the *target constructor*. ~~The *principal constructor* is the first constructor invoked in the construction of an object (that is, not a target constructor for that object's construction).~~ The target constructor is selected by overload resolution. ...

Change in 12.6.3 [class.inhctor.init] paragraph 3:

> When an object is initialized by an ~~inheriting~~ **inherited** constructor, the ~~principal constructor (12.6.2 [class.base.init], 15.2 [except.ctor]) for the object is considered to have completed execution~~ **initialization of the object is complete** when the initialization of all subobjects is complete.

Change in 15.2 [except.ctor] paragraphs 3 and 4:

> ~~For an object of class type of any storage duration whose~~ **If the** initialization or destruction **of an object other than by a delegating constructor** is terminated by an exception, the destructor is invoked for each of the object's ~~fully constructed~~ **direct subobjects and, for a complete object, virtual base class** subobjects, ~~that is, for each subobject for which the principal constructor (12.6.2)has completed execution~~ **whose initialization has completed (8.6 [dcl.init])** and **whose** ~~the~~ destructor has not yet begun execution, except that in the case of destruction, the variant members of a union-like class are not destroyed. The subobjects are destroyed in the reverse order of the completion of their construction. Such destruction is sequenced before entering a handler of the function-try-block of the constructor or destructor, if any.
>
> ~~Similarly, if the principal constructor for an object has completed execution and a delegating constructor for that object exits with an exception, the object's destructor is invoked.~~ **If the *compound-statement* of the *function-body* of a delegating constructor for an object exits via an exception, the object's destructor is invoked.** Such destruction is sequenced before entering a *handler* of the *function-try-block* of a delegating constructor for that object, if any.

US 93: argument evaluation order when calling an operator function

Change in 5.2.2 [expr.call] paragraph 5:

> The *postfix-expression* is sequenced before each expression in the *expression-list* and any default argument. The

initialization of a parameter, including every associated value computation and side effect, is indeterminately sequenced with respect to that of any other parameter. [Note: All side effects of argument evaluations are sequenced before the function is entered (see 1.9 [intro.execution]). -- end note] [Example: ...] **[Note: If an operator function is invoked using operator notation, argument evaluation is sequenced as specified for the built-in operator; see 13.3.1.2 [over.match.oper]. -- end note] [Example:**

```
struct S {
  S(int);
};
int operator<<(S, int);
int i;
int x = S(i=1) << (i=2);
int j;
int y = operator<<(S(j=1), j=2);
```

After performing the initializations, the value of `i` is 2 (see 5.8 [expr.shift]), but it is unspecified whether the value of `j` is 1 or 2.-- end example]

Drafting note: There is no expression-list when using operator notation.

US 94, GB 13, FI 21: class template deduction in explicit type conversion using a *braced-init-list*

Change in 5.2.3 [expr.type.conv] paragraphs 1 and 2:

A *simple-type-specifier* (7.1.7.2) or *typename-specifier* (14.6) followed by a parenthesized optional *expression-list* or by a *braced-init-list* (the initializer) constructs a value of the specified type given the initializer. **If the type is a placeholder for a deduced class type, it is replaced by the return type of the function selected by overload resolution for class template deduction (13.3.1.8 [over.match.class.deduct]) for the remainder of this section.**

If the initializer is a parenthesized single expression, the type conversion expression is equivalent (in definedness, and if defined in meaning) to the corresponding cast expression (5.4). If the type is (possibly cv-qualified) void and the initializer is (), the expression is a prvalue of the specified type that performs no initialization. Otherwise, the expression is a prvalue of the specified type whose result object is direct-initialized (8.6) with the initializer. For an expression of the form T(), T shall not be an array type.

~~A *template-name* corresponding to a class template followed by a parenthesized *expression-list* constructs a value of a particular type determined as follows. Given such an expression `T(x1, x2, ...)`, construct the declaration `T t(x1, x2, ...)`; for some invented variable t. Define `U` to be `decltype(t)`, then the expression `T(x1, x2, ...)` is equivalent to the expression `U(x1, x2, ...)`.~~

Drafting note: Since the braced-init-list syntax for types (as opposed to template-names) was already valid, and there is a syntax transformation to a variable declaration, no additional ambiguities are apparent by allowing the additional context for class template deduction.

US 103: explicit deduction guides consider default template arguments

Drafting note: 14.3 [temp.arg] p7 seems to be clear that default arguments are considered when parsing a template-id. Given the fact that 14.9 [temp.deduct.guide] is devoid of any example right now, it seems a good idea to add one.

Add a new paragraph after 14.9 [temp.deduct.guide] paragraph 1:

[Example:

```
template<class T, class D = int>
struct S {
  T data;
};
template<class U>
S(U) -> S<typename U::type>;

struct A {
  using type = short;
  operator type();
};
S x{A()};  // x is of type S<short, int>
```

-- end example]

GB 5: definition of template parameter

This is a defect against C++14.

Change in 1.3.17 [defns.parameter.templ]:

> **parameter**
> <template> ~~*template-parameter*~~ **member of a *template-parameter-list***

GB 6: definition of undefined behavior vs. constexpr

This is a defect against C++14.

Change in 1.3.25 [defns.undefined]:

> **undefined behavior**
> behavior for which this International Standard imposes no requirements [Note: Undefined behavior may be expected when this International Standard omits any explicit definition of behavior or when a program uses an erroneous construct or erroneous data. Permissible undefined behavior ranges from ignoring the situation completely with unpredictable results, to behaving during translation or program execution in a documented manner characteristic of the environment (with or without the issuance of a diagnostic message), to terminating a translation or execution (with the issuance of a diagnostic message). Many erroneous program constructs do not engender undefined behavior; they are required to be diagnosed. **Evaluation of a constant expression never exhibits behavior explicitly specified as undefined (5.20 [expr.const]).** -- end note]

Add at the end of 1.4 [intro.compliance] paragraph 4:

- ...

[Note: During template argument deduction and substitution, certain constructs that in other contexts require a diagnostic are treated differently; see 14.8.2 [temp.deduct]. -- end note]

GB 19: missing cv-qualification when materializing a temporary object

Change in 8.6.3 [dcl.init.ref] paragraph 5.2.3:

- If the initializer expression
  - is an rvalue (but not a bit-field) or function lvalue and "cv1 T1" is reference-compatible with "cv2 T2", or
  - has a class type (i.e., T2 is a class type), where T1 is not reference-related to T2, and can be converted to an rvalue of type "cv3 T3", where "cv1 T1" is reference-compatible with "cv3 T3" (see 13.3.1.6 [over.match.ref]),

  then ~~the reference is bound to~~ the value of the initializer expression in the first case and ~~to~~ the result of the conversion in the second case **is called the converted initializer. If the converted initializer is a prvalue, its type `T4` is adjusted to type `cv1 T4` (4.5 [conv.qual]) and the temporary materialization conversion (4.4 [conv.rval]) is applied. In any case, the reference is bound to the resulting glvalue** (or~~, in either case,~~ to an appropriate base class subobject) ~~after applying the temporary materialization conversion (4.4 [conv.rval])~~. [Example: ...]

GB 20: decomposition declaration should commit to tuple interpretation early

Change in 8.5 [dcl.decomp] paragraph 3:

> Otherwise, if **the *qualified-id* `std::tuple_size<E>` names a complete type,** the expression `std::tuple_size<E>::value` ~~is~~ **shall be** a well-formed integral constant expression~~;~~ **and** the number of elements in the *identifier-list* shall be equal to the value of that expression. ...

GB 23: handlers taking a reference to array or function

This is a defect against C++14.

Change in 15.3 [except.handle] paragraph 3:

> [Note: A *throw-expression* whose operand is an integer literal with value zero does not match a handler of pointer or pointer to member type. **A handler of reference to array or function type is never a match for any exception object (5.17 [expr.throw]).** -- end note]

GB 25: imprecise description when terminate is called

This is a defect against C++14.

Change in 15.1 [except.throw] paragraph 7:

> If the exception handling mechanism~~, after completing the initialization of the exception object but before the activation of a handler for the exception, calls~~ **handling an uncaught exception (15.5.3 [except.uncaught]) directly invokes** a function that exits via an exception, `std::terminate` is called (15.5.1 [except.terminate]). [Example: ...] **[Note: Consequently, destructors should generally catch exceptions and not let them propagate.]**

Change in 15.2 [except.ctor] paragraph 1:

> ... ~~If a destructor directly invoked by stack unwinding exits with an exception, std::terminate is called (15.5.1). [Note: Consequently, destructors should generally catch exceptions and not let them propagate out of the destructor. -- end note]~~

GB 26: 'last' active handler and multithreading

This is a defect against C++14.

Change in 15.1 [except.throw] paragraph 4:

> The memory for the exception object is allocated in an unspecified way, except as noted in 3.7.4.1 [basic.stc.dynamic.allocation]. If a handler exits by rethrowing, control is passed to another handler for the same exception **object**. The ~~exception object is destroyed after either the last remaining~~ **points of potential destruction for the exception object are:**
>
> - **an** active handler for the exception exits by any means other than rethrowing~~, or the last~~ **, immediately after the destruction of the object (if any) declared in the *exception-declaration* in the handler;**
> - **an** object of type `std::exception_ptr` (18.8.6 [propagation]) that refers to the exception object is destroyed, **before the destructor of `std::exception_ptr` returns** ~~whichever is later~~.
>
> ~~In the former case, the destruction occurs when the handler exits, immediately after the destruction of the object declared in the exception-declaration in the handler, if any. In the latter case, the destruction occurs before the destructor of `std::exception_ptr` returns.~~ **Among all points of potential destruction for the exception object, there is an unspecified last one where the exception object is destroyed. All other points happen before that last one (1.10.1 [intro.races]). [Note: No other thread synchronization is implied in exception handling.]** The implementation may then deallocate the memory for the exception object; any such deallocation is done in an unspecified way. [Note: a thrown exception does not propagate to other threads unless caught, stored, and rethrown using appropriate library functions; see 18.8.6 and 30.6. -- end note]

GB 27: multiple activations of the same exception object

This is a defect against C++14.

Change in 15.5.3 [except.uncaught] paragraph 1:

> An exception is considered uncaught after completing the initialization of the exception object (15.1 [except.throw]) until completing the activation of a handler for the exception (15.3 [except.handle]). This includes stack unwinding. If ~~the~~ **an** exception is rethrown (5.17**, 18.8.6 [propagation]**), it is considered uncaught from the point of rethrow until the rethrown exception is caught ~~again~~. The function std::uncaught_exceptions() (18.8.5 [uncaught.exceptions]) returns the number of uncaught exceptions in the current thread.

GB 63: add limit for number of lambda-captures

This is a defect against C++14.

Add in Annex B [implimits]:

> - **Lambda-captures in one *lambda-expression* [256].**

GB 64: add limit for length of initializer-list

This is a defect against C++14.

Add in Annex B [implimits]:

- **Initializer-clauses in one *braced-init-list* [16384].**

GB 65: add limit for number of identifiers in decomposition declaration

Add in Annex B [implimits]:

- **Identifiers introduced in one decomposition declaration [256].**

FI 20: decomposition declarations with parentheses

Change in 7 [dcl.dcl]:

simple-declaration:
decl-specifier-seq init-declarator-list$_{opt}$;
attribute-specifier-seq decl-specifier-seq init-declarator-list ;
attribute-specifier-seq$_{opt}$ *decl-specifier-seq ref-qualifier*$_{opt}$
[identifier-list] ~~*brace-or-equal-initializer*~~ ***initializer*** ;

Change in 7 [dcl.dcl] paragraph 8:

The ~~*brace-or-equal-initializer*~~ ***initializer*** shall be of the form "= *assignment-expression*" ~~or~~ **,** of the form "{ *assignment-expression* }", **or of the form "(*assignment-expression*)",** where the *assignment-expression* is of array or non-union class type.

Change in 8.5 [dcl.decomp] paragraph 1:

If the *assignment-expression* in the ~~*brace-or-equal-initializer*~~ ***initializer*** has array type A and no *ref-qualifier* is present, e has type cv A and each element is copy-initialized or direct-initialized from the corresponding element of the *assignment-expression* as specified by the form of the ~~*brace-or-equal-initializer*~~ ***initializer***. Otherwise, e is defined as-if by

attribute-specifier-seq $_{opt}$ *decl-specifier-seq ref-qualifier*$_{opt}$ *e* ~~*brace-or-equal-initializer*~~ ***initializer*** ;

where **the declaration is never interpreted as a function declaration and** the parts of the declaration other than the *declarator-id* are taken from the corresponding decomposition declaration. ...

RU 1: CWG 253: Why must empty or fully-initialized const objects be initialized?

This is a defect against C++14.

Change in 8.6 [dcl.init] paragraph 7:

To *default-initialize* an object of type T means:

- ...
- ...
- ...

A class type T is *const-default-constructible* if default-initialization of T would invoke a user-provided constructor of T (not inherited from a base class) or if

- **each direct non-variant non-static data member M of T has a default member initializer or, if M is of class type X (or array thereof), X is const-default-constructible,**
- **if T is a union with at least one non-static data member, exactly one variant member has a default member initializer,**
- **if T is a not a union, for each anonymous union member with at least one non-static data member (if any), exactly one non-static data member has a default member initializer, and**
- **each potentially constructed base class of T is const-default-constructible.**

If a program calls for the default-initialization of an object of a const-qualified type T, T shall be a **const-default-constructible** class type **or array thereof** ~~with a user-provided default constructor~~.